Bazele Inteligenţei Artificiale

Prof. Dr. Călin-Adrian POPA

Cursul 2

7 Octombrie 2025

2 Nonparametric models

- linear models use the training data to estimate a fixed set of parameters $\boldsymbol{w}$
- after the parameters $\boldsymbol{w}$ have been established, we can throw away the training data, because they are all summarized by $\boldsymbol{w}$
- a learning model that summarizes data with a set of parameters of fixed size (independent of the number of training examples) is called a *parametric model*
- a *nonparametric model* is one that cannot be characterized by a bounded set of parameters; they may retain part or all of the data points as part of the model
- learning methods that do this have also been described as *instance-based learning* or *memory-based learning*

2 Nonparametric models

- the simplest instance-based learning method is *table lookup*: take all the training examples, put them in a lookup table, and then, when asked for the prediction for $\boldsymbol{x}$, see if $\boldsymbol{x}$ is in the table; if it is, return the corresponding y
- the problem with this method is that it does not generalize well: when $\boldsymbol{x}$ is not in the table, we have no information about a plausible value
- we can improve on table lookup with a slight variation: given a query $\boldsymbol{x}^{(t)}$, instead of finding an example that is equal to $\boldsymbol{x}^{(t)}$, find the k examples that are nearest to $\boldsymbol{x}^{(t)}$
- this is called *k-nearest neighbors* lookup

2.1 *k*-Nearest neighbors

- the *k-nearest neighbors* algorithm is arguably the simplest machine learning algorithm
- building the model consists only of storing the training dataset
- to make a prediction for a new data point, the algorithm finds the closest data points in the training dataset – its “nearest neighbors”
- in its simplest version, the *k*-nearest neighbors algorithm only considers exactly one nearest neighbor, which is the closest training data point to the point we want to make a prediction for
- the prediction is then simply the known output for this training point
- Figure 1 illustrates this for the case of classification on the `forge` dataset

2.1 k-Nearest neighbors

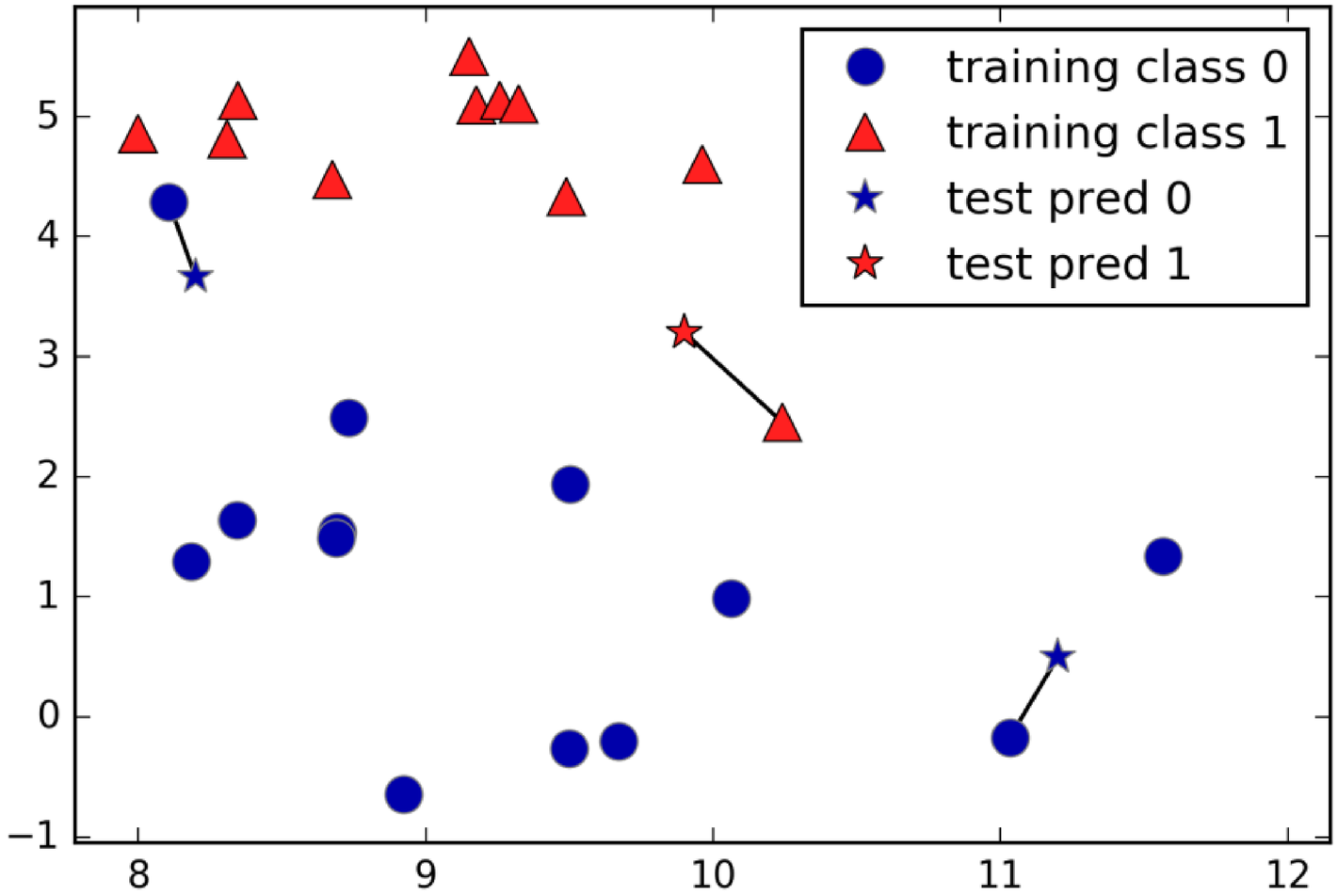


Figure 1: Predictions made by the one-nearest neighbor model on the `forge` dataset.

2.1 k-Nearest neighbors

- here, we added three new data points, shown as stars
- for each of them, we marked the closest point in the training set
- the prediction of the one-nearest neighbor algorithm is the label of that point (shown by the color of the star)

2.1 k-Nearest neighbors

- instead of considering only the closest neighbor, we can also consider an arbitrary number, k, of neighbors
- this is where the name of the k-nearest neighbors algorithm comes from
- when considering more than one neighbor, we use *voting* to assign a label
- this means that for each test point, we count how many neighbors belong to class 0 and how many neighbors belong to class 1
- we then assign the class that is more frequent: in other words, the *majority* class among the k-nearest neighbors
- the example in Figure 2 uses the three closest neighbors

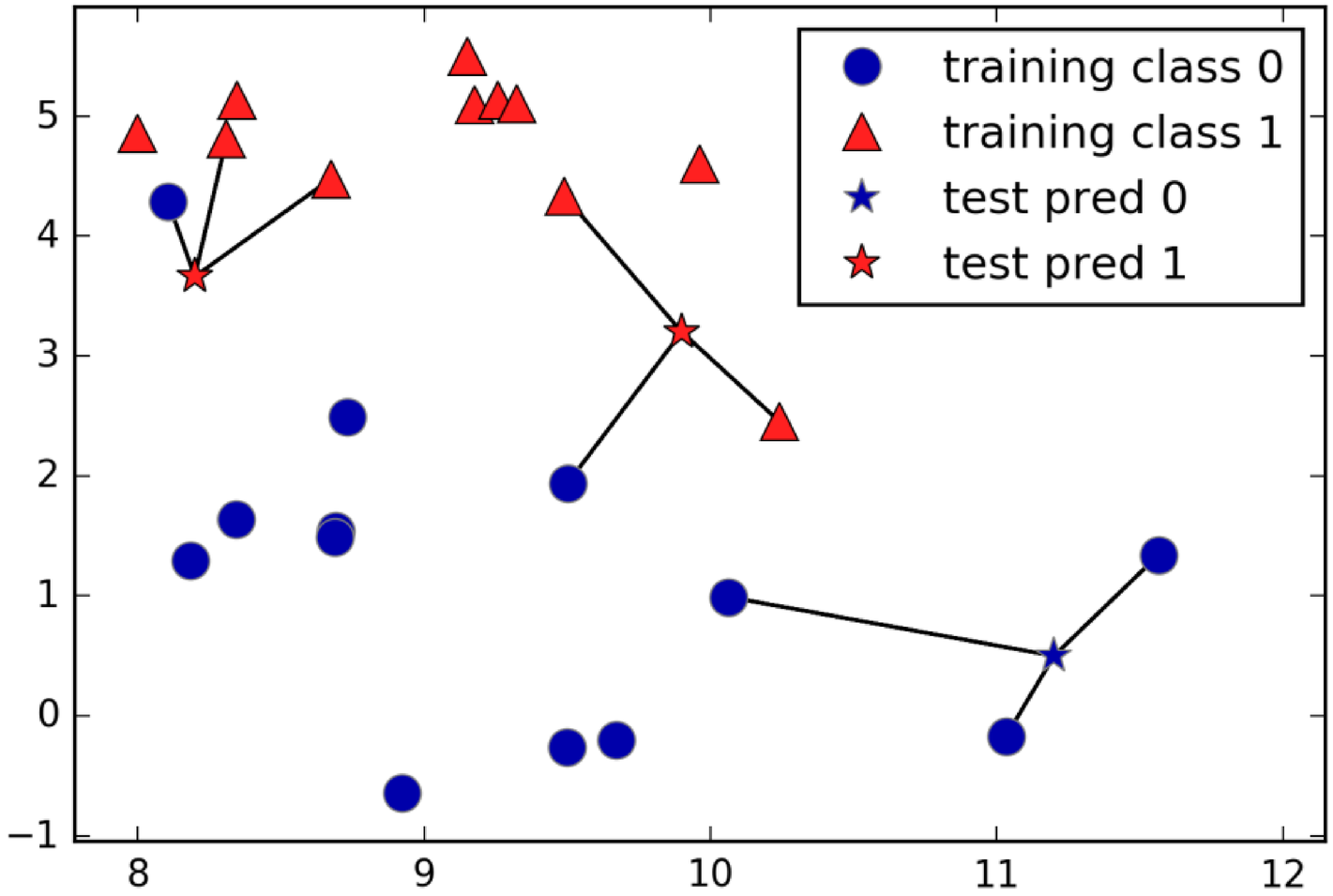


Figure 2: Predictions made by the three-nearest neighbors model on the `forge` dataset.

2.1 *k*-Nearest neighbors

- again, the prediction is shown as the color of the star; we can see that the prediction for the new data point at the top left is not the same as the prediction when we used only one neighbor
- while this illustration is for a binary classification problem, this method can be applied to datasets with *any number of classes*
- for more classes, we count how many neighbors belong to each class and again predict the most common class
- for two-dimensional datasets, we can also illustrate the prediction for all possible test points in the *xy*-plane
- we color the plane according to the class that would be assigned to a point in this region
- this lets us view the *decision boundary* (Figure 3), which is the divide between where the algorithm assigns class 0 versus where it assigns class 1

2.1 k-Nearest neighbors

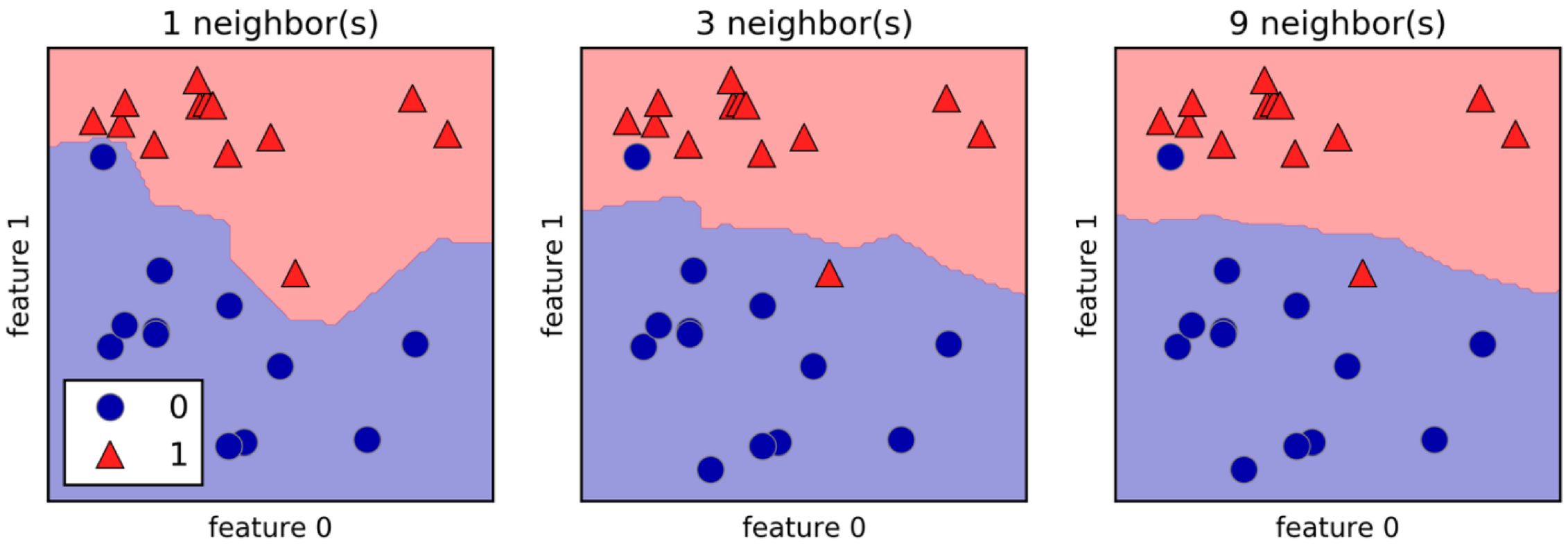


Figure 3: Decision boundaries created by the nearest neighbors model for different values of the number of neighbors.

2.1 *k*-Nearest neighbors

- as we can see on the left in Figure 3, using a single neighbor results in a decision boundary that follows the training data closely
- considering more and more neighbors leads to a smoother decision boundary
- a smoother boundary corresponds to a simpler model; in other words, using *few neighbors* corresponds to *high* model *complexity*, and using *many neighbors* corresponds to *low* model *complexity*
- if we consider the extreme case where the number of neighbors is the number of all data points in the training set, each test point would have exactly the same neighbors (all training points) and all predictions would be the same: the class that is most frequent in the training set

2.1 k-Nearest neighbors

- there is also a regression variant of the k-nearest neighbors algorithm
- again, let's start by using the single nearest neighbor, this time on the `wave` dataset
- we have added three test data points as green stars on the x-axis
- the prediction using a single neighbor is just the target value of the nearest neighbor
- these are shown as blue stars in Figure 4

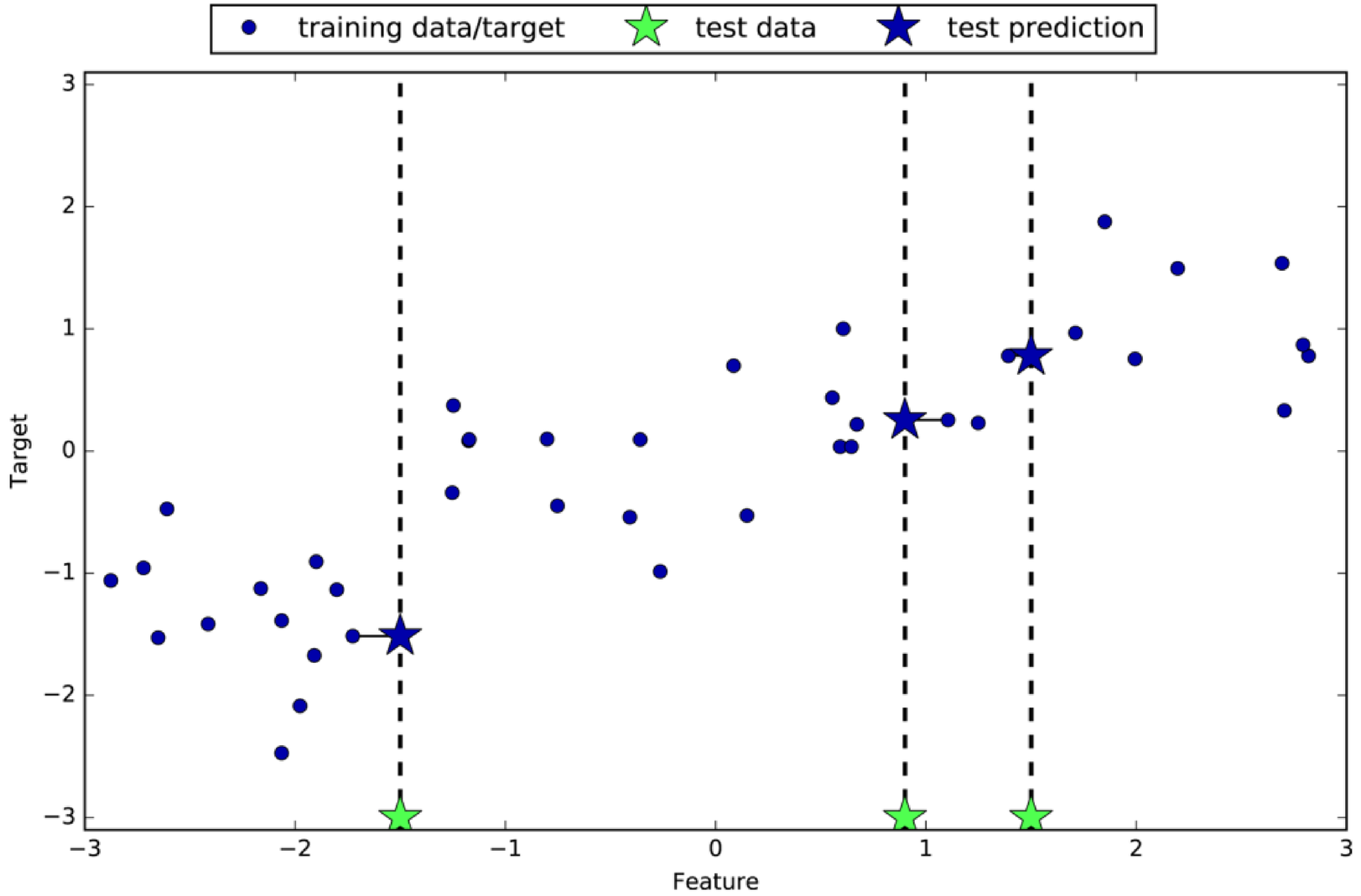


Figure 4: Predictions made by one-nearest neighbor regression on the `wave` dataset.

2.1 *k*-Nearest neighbors

- again, we can use more than the single closest neighbor for regression
- when using multiple nearest neighbors, the prediction is the *average*, or *mean*, of the relevant neighbors (Figure 5)

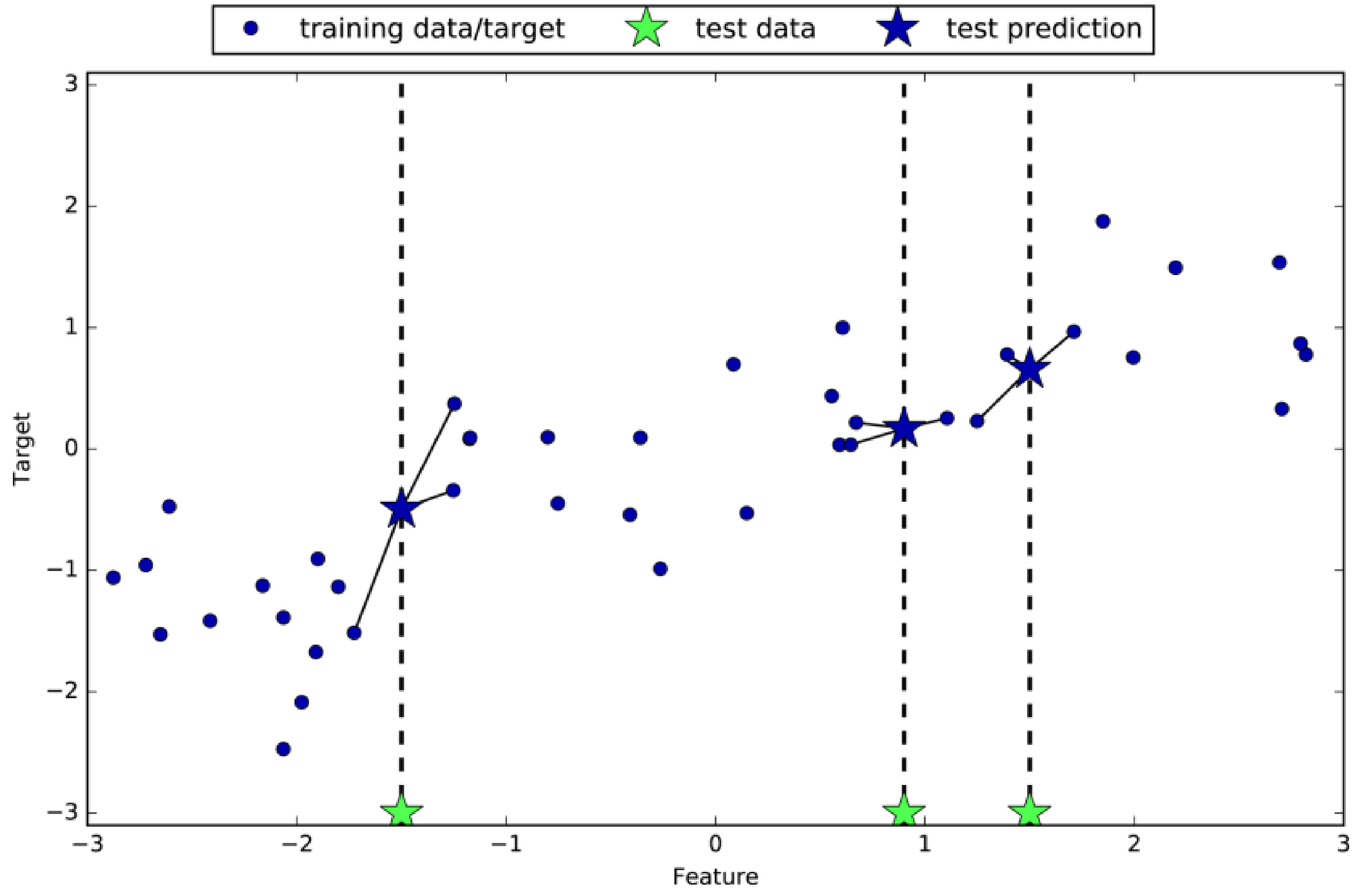


Figure 5: Predictions made by three-nearest neighbors regression on the `wave` dataset.

2.1 *k*-Nearest neighbors

- in principle, there are two important parameters to the *k*-nearest neighbors classifier: the *number of neighbors* and how we measure *distance* between data points
- in practice, using a small number of neighbors like three or five often works well, but we should certainly adjust this parameter
- with regard to the distance measure, by default, *Euclidean distance* is used, which works well in many settings

2.1 k-Nearest neighbors

- possible distance variants are the *Minkowski distance* or ℓ_p-norm; with $p = 2$ this is *Euclidean distance* and with $p = 1$ it is *Manhattan distance*
- often Euclidean distance is used if the dimensions are measuring similar properties, such as the width, height, and depth of parts, and Manhattan distance is used if they are dissimilar, such as age, weight, and gender of a patient
- how do we compare a difference in age to a difference in weight?
- a common approach is to apply *normalization* to the measurements in each dimension
- a more complex metric, known as the *Mahalanobis distance*, takes into account the *covariance* between dimensions
- with boolean attribute values, the number of attributes on which two points differ is called the *Hamming distance*

2.1 *k*-Nearest neighbors

- one of the strengths of *k*-nearest neighbors is that the model is very easy to understand, and often gives reasonable performance without a lot of adjustments
- using this algorithm is a good baseline method to try before considering more advanced techniques
- building the nearest neighbors model is usually very fast, but when the training set is very large (either in number of features or in number of examples) prediction can be slow
- when using the *k*-nearest neighbors algorithm, it's important to preprocess the data
- this approach often does not perform well on datasets with many features (hundreds or more), and it does particularly badly with datasets where most features are 0 most of the time (so-called *sparse datasets*)

2.2 Linear SVM classification

- a *support vector machine (SVM)* is a powerful and versatile machine learning model, capable of performing linear or nonlinear classification, regression, and even outlier detection
- it is one of the most popular models in machine learning, and anyone interested in machine learning should have it in their toolbox
- SVMs are particularly well suited for classification of complex small- or medium-sized datasets
- first, we consider *binary classification*, for which the set of possible values that the label/output can attain is binary
- traditionally, SVMs use the convention that class labels are $+1$ and -1 for the *positive* and *negative classes*, respectively, instead of the $+1$ and 0 we have been using so far

2.2 Linear SVM classification

- suppose we have a set of inputs $\boldsymbol{x}^{(i)} \in \mathbb{R}^d$, $1 \leq i \leq n$, along with their corresponding (binary) targets $y^{(i)} \in \{+1, -1\}$, $1 \leq i \leq n$
- given a training dataset consisting of examples, we would like to estimate parameters of the model that will give the smallest classification error
- we derive the optimization problem corresponding to training an SVM
- intuitively, we imagine binary classification data, which can be separated by a hyperplane, as illustrated in Figure 6

2.2 Linear SVM classification

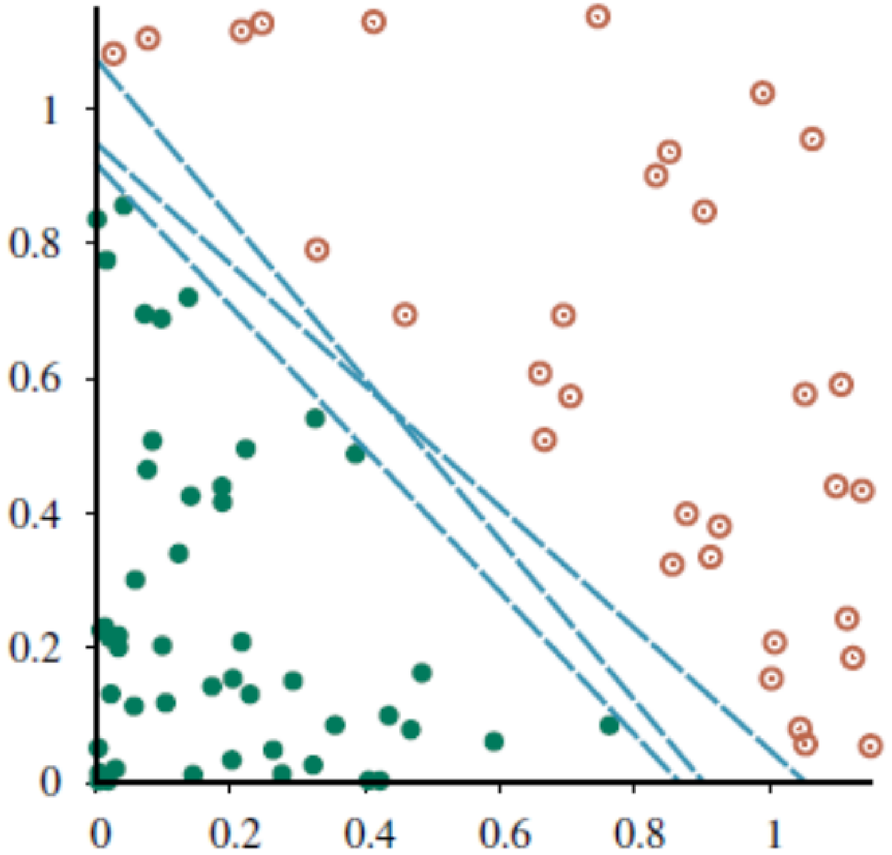


Figure 6: Two classes of points (orange open and green filled circles) and three candidate linear separators.

2.2 Linear SVM classification

- here, every example $\boldsymbol{x}^{(i)}$ (a vector of dimension 2) is a two-dimensional location ($x_1^{(i)}$ and $x_2^{(i)}$), and the corresponding binary label $y^{(i)}$ is one of two different symbols (orange open and green filled circles)
- the examples consist of two classes (there are two possible labels) that have features (the components of the vector representing the example) arranged in such a way as to allow us to separate/classify them by drawing a straight line
- we say that the two classes are *linearly separable*

2.2 Linear SVM classification

- the main idea behind many classification algorithms is to represent data in $\mathbb{R}^d$ and then partition this space, ideally in a way that examples with the same label (and no other examples) are in the same partition
- in the case of binary classification, the space would be divided into two parts corresponding to the positive and negative classes, respectively
- we consider a particularly convenient partition, which is to (linearly) split the space into two halves using a *hyperplane*
- a hyperplane is an affine subspace of dimension $d - 1$ (if the corresponding vector space is of dimension d)
- whereas we previously put the intercept into the weight vector $\boldsymbol{w}$ (and a corresponding dummy 1 value into x_0), SVMs do not do that; they keep the intercept as a separate parameter, w_0
- thus, the hyperplane that separates the two classes in our binary classification problem is defined as the set of points:

$$\{\boldsymbol{x} \in \mathbb{R}^d | \boldsymbol{w}^\top \boldsymbol{x} + w_0 = 0\}. \tag{1}$$

2.2 Linear SVM classification

- when presented with a test example, we classify the example as positive or negative depending on the side of the hyperplane on which it occurs
- note that (1) not only defines a hyperplane, it additionally defines a direction; in other words, it defines the positive and negative side of the hyperplane
- therefore, to classify a test example $\boldsymbol{x}^{(t)}$, we calculate the value of the function $\boldsymbol{w}^\top \boldsymbol{x}^{(t)} + w_0$ and classify the example as $+1$ if $\boldsymbol{w}^\top \boldsymbol{x}^{(t)} + w_0 \geq 0$ and -1, otherwise
- thinking geometrically, the positive examples lie “above” the hyperplane and the negative examples “below” the hyperplane

2.2 Linear SVM classification

- when training the classifier, we want to ensure that the examples with positive labels are on the positive side of the hyperplane:

$$\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0 \geq 0 \text{ when } y^{(i)} = +1,$$

and the examples with negative labels are on the negative side:

$$\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0 < 0 \text{ when } y^{(i)} = -1,$$

for $1 \leq i \leq n$

- these two conditions are often presented in a single equation:

$$y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq 0. \tag{2}$$

2.2 Linear SVM classification

- for a dataset that is linearly separable, we have infinitely many candidate hyperplanes, and therefore classifiers, that solve our classification problem without any (training) errors
- this is because a given separating hyperplane can usually be shifted a tiny bit up or down, or rotated, without coming into contact with any of the examples
- three possible separating hyperplanes are shown in Figure 6
- consider the lowest of the three separating lines in Figure 6; it comes very close to five of the green filled circles examples
- although it classifies all the examples correctly, and thus minimizes loss, it's probably not a good thing that so many examples are close to the line; it may be that other green filled circles examples will turn out to fall on the wrong side of the line

2.2 Linear SVM classification

- in order to construct a classifier based upon a separating hyperplane, we must have a reasonable way to decide which of the infinite possible separating hyperplanes to use
- to find a unique solution, one idea is to choose the separating hyperplane that maximizes the margin between the positive and negative examples
- in other words, we want the positive and negative examples to be separated by the *largest possible margin*, i.e., to be as far away from the closest training examples as possible
- we call this separator, shown in Figure 7, the *maximum margin separator* or *maximum margin hyperplane*
- the *margin* is the width of the area bounded by dashed lines in the figure – twice the distance from the separator to the nearest example point

2.2 Linear SVM classification

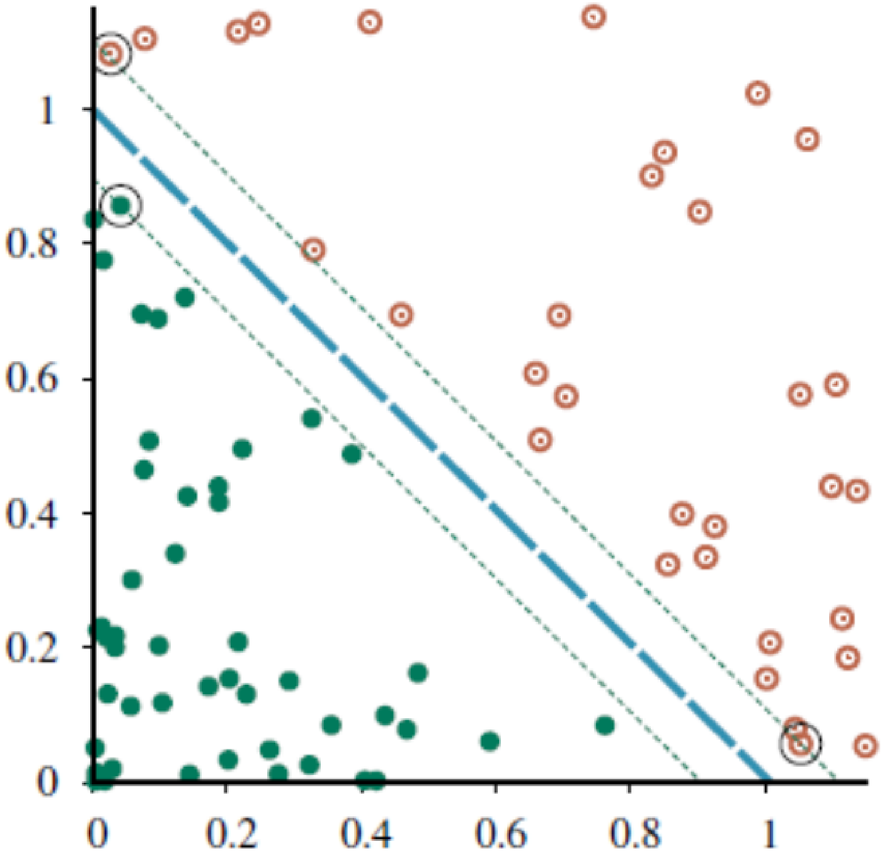


Figure 7: The *maximum margin separator* (heavy line) is at the midpoint of the *margin* (area between dashed lines). The *support vectors* (points with large circles) are the examples closest to the separator; here there are three.

2.2 Linear SVM classification

- examining Figure 7, we see that three training examples are equidistant from the maximum margin hyperplane and lie along the dashed lines indicating the width of the margin
- these three examples are known as *support vectors*, since they are vectors in d-dimensional space (in Figure 7, $d = 2$) and they "support" the maximum margin hyperplane in the sense that if these points were moved slightly, then the maximum margin hyperplane would move as well
- interestingly, the maximum margin hyperplane depends directly on the support vectors, but not on the other examples: a movement to any of the other examples would not affect the separating hyperplane, provided that the example's movement does not cause it to cross the boundary set by the margin
- the fact that the maximum margin hyperplane depends directly on only a small subset of the examples is an important property of linear SVMs, as opposed to linear models, which depend on all the example points

2.2 Linear SVM classification

- the concept of the *margin* is intuitively simple: it is the distance of the separating hyperplane to the closest examples in the dataset, assuming that the dataset is linearly separable
- however, when trying to formalize this distance, there is a technical problem
- the technical problem is that we need to define a scale at which to measure the distance
- a potential scale is to consider the scale of the data, i.e., the raw values of $\boldsymbol{x}^{(i)}$

2.2 Linear SVM classification

- consider a hyperplane $\boldsymbol{w}^\top \boldsymbol{x} + w_0$, and an example $\boldsymbol{x}^{(a)}$
- without loss of generality, we can consider the example $\boldsymbol{x}^{(a)}$ to be on the positive side of the hyperplane, i.e., $\boldsymbol{w}^\top \boldsymbol{x}^{(a)} + w_0 > 0$
- we would like to compute the distance $r > 0$ of $\boldsymbol{x}^{(a)}$ from the hyperplane
- we do so by considering the orthogonal projection of $\boldsymbol{x}^{(a)}$ onto the hyperplane, which we denote by $\boldsymbol{x}_\perp^{(a)}$
- since $\boldsymbol{w}$ is orthogonal to the hyperplane, we know that the distance r is just a scaling of this vector $\boldsymbol{w}$

2.2 Linear SVM classification

- if the length of $\boldsymbol{w}$ is known, then we can use this scaling factor r to work out the absolute distance between $\boldsymbol{x}^{(a)}$ and $\boldsymbol{x}_{\perp}^{(a)}$
- for convenience, we choose to use a vector of unit length (its norm is 1) and obtain this by dividing $\boldsymbol{w}$ by its norm, $\frac{\boldsymbol{w}}{||\boldsymbol{w}||_2}$
- using vector addition, we obtain:

$$\boldsymbol{x}^{(a)} = \boldsymbol{x}_{\perp}^{(a)} + r\frac{\boldsymbol{w}}{||\boldsymbol{w}||_2}. \tag{3}$$

2.2 Linear SVM classification

- we have now expressed the distance of $\boldsymbol{x}^{(a)}$ from the hyperplane as r, and if we choose $\boldsymbol{x}^{(a)}$ to be the point closest to the hyperplane, this distance r is the *margin*
- recall that we would like the positive examples to be further than r from the hyperplane, and the negative examples to be further than distance r (in the negative direction) from the hyperplane
- analogously to (2), we formulate this objective as:

$$y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq r. \tag{4}$$

- in other words, we combine the requirements that examples are at least r away from the hyperplane (in the positive and negative direction) into one single inequality

2.2 Linear SVM classification

- since we are interested only in the direction, we add an assumption to our model that the parameter vector $\boldsymbol{w}$ is of unit length, i.e., $||\boldsymbol{w}||_2 = 1$
- this assumption also allows a more intuitive interpretation of the distance r, since it is the scaling factor of a vector of length 1
- collecting the three requirements into a single constrained optimization problem, we obtain the objective:

$$\begin{aligned} &\max_{\boldsymbol{w}, w_0, r} \; r \\ &\text{subject to } y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq r, \; 1 \leq i \leq n, \; ||\boldsymbol{w}||_2 = 1, \; r > 0, \end{aligned} \tag{5}$$

which says that we want to *maximize* the *margin* r, while ensuring that the data lies on the *correct* side of the hyperplane

2.2 Linear SVM classification

- we now derive the margin maximization problem by making a different assumption
- instead of choosing that the parameter vector is normalized, we choose a scale for the data
- we choose this scale such that the value of the predictor $\boldsymbol{w}^\top \boldsymbol{x} + w_0$ is 1 at the closest example
- let us also denote the example in the dataset that is closest to the hyperplane by $\boldsymbol{x}^{(a)}$
- the example $\boldsymbol{x}^{(a)}$ lies exactly on the margin, i.e., $\boldsymbol{w}^\top \boldsymbol{x}^{(a)} + w_0 = 1$
- since $\boldsymbol{x}_\perp^{(a)}$ is the orthogonal projection of $\boldsymbol{x}^{(a)}$ onto the hyperplane, it must by definition lie on the hyperplane, i.e.:

$$\boldsymbol{w}^\top \boldsymbol{x}_\perp^{(a)} + w_0 = 0. \tag{6}$$

2.2 Linear SVM classification

- by substituting (3) into (6), we obtain:

$$\begin{aligned}
\boldsymbol{w}^\top \left(\boldsymbol{x}^{(a)} - r\frac{\boldsymbol{w}}{||\boldsymbol{w}||_2} \right) + w_0 = 0 \quad &\Leftrightarrow \quad \boldsymbol{w}^\top \boldsymbol{x}^{(a)} - r\frac{\boldsymbol{w}^\top \boldsymbol{w}}{||\boldsymbol{w}||_2} + w_0 = 0 \\
&\Leftrightarrow \quad \boldsymbol{w}^\top \boldsymbol{x}^{(a)} + w_0 - r\frac{||\boldsymbol{w}||_2^2}{||\boldsymbol{w}||_2} = 0 \\
&\Leftrightarrow \quad 1 - r||\boldsymbol{w}||_2 = 0 \\
&\Leftrightarrow \quad r = \frac{1}{||\boldsymbol{w}||_2},
\end{aligned}$$

where, by our assumption, $\boldsymbol{w}^\top \boldsymbol{x}^{(a)} + w_0 = 1$

2.2 Linear SVM classification

- this means we derived the distance r in terms of the normal vector $\boldsymbol{w}$ of the hyperplane
- at first glance, this equation is counterintuitive, as we seem to have derived the distance from the hyperplane in terms of the length of the vector $\boldsymbol{w}$, but we do not yet know this vector
- one way to think about it is to consider the distance r to be a temporary variable that we only use for this derivation
- therefore, we will denote the distance to the hyperplane by $\frac{1}{||\boldsymbol{w}||_2}$
- similar to the argument to obtain (4), we want the positive and negative examples to be at least 1 away from the hyperplane, which yields the condition:

$$y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq 1.$$

2.2 Linear SVM classification

- combining the margin maximization with the fact that examples need to be on the correct side of the hyperplane (based on their labels) gives us the objective:

$$\begin{aligned} &\max_{\boldsymbol{w}, w_0} \frac{1}{||\boldsymbol{w}||_2} \\ &\text{subject to } y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq 1,\ 1 \leq i \leq n. \end{aligned} \tag{7}$$

- instead of maximizing the reciprocal of the norm as in (7), we often minimize the squared norm, because its gradient is easier to compute
- we also often include a constant $\frac{1}{2}$ that does not affect the optimal $\boldsymbol{w}, w_0$ but yields a tidier form when we compute the gradient
- then, our objective becomes:

$$\begin{aligned} &\min_{\boldsymbol{w}, w_0} \frac{1}{2}||\boldsymbol{w}||_2^2 \\ &\text{subject to } y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq 1,\ 1 \leq i \leq n. \end{aligned} \tag{8}$$

2.2 Linear SVM classification

- equation (8) is known as the *hard margin SVM*
- the reason for the hard margin SVM expression “hard” is because the formulation does not allow for any *violations* of the margin condition
- on the one hand, we argued that we would like to maximize some value r, which represents the distance of the closest example to the hyperplane
- on the other hand, we scaled the data such that the closest example is of distance 1 to the hyperplane
- now we relate the two derivations, and show that they are equivalent

2.2 Linear SVM classification

- consider (5); since the square is a strictly monotonic transformation for non-negative arguments, the maximum stays the same if we consider r^2 in the objective
- since $||\boldsymbol{w}||_2 = 1$, we can reparametrize the equation with a new weight vector $\boldsymbol{w}'$ that is not normalized by explicitly using $\frac{\boldsymbol{w}'}{||\boldsymbol{w}'||_2}$; we obtain:

$$\max_{\boldsymbol{w}', w_0, r} \; r^2$$
$$\text{subject to } y^{(i)} \left(\frac{\boldsymbol{w}'^{\top}}{||\boldsymbol{w}'||_2} \boldsymbol{x}^{(i)} + w_0 \right) \geq r, \; 1 \leq i \leq n, \; r > 0. \tag{9}$$

- equation (9) explicitly states that the distance r is positive
- therefore, we can divide the constraint by r, which yields:

$$\max_{\boldsymbol{w}', w_0, r} \; r^2$$
$$\text{subject to } y^{(i)} \left(\frac{\boldsymbol{w}'^{\top}}{||\boldsymbol{w}'||_2 r} \boldsymbol{x}^{(i)} + \frac{w_0}{r} \right) \geq 1, \; 1 \leq i \leq n, \; r > 0. \tag{10}$$

2.2 Linear SVM classification

- we rename the parameters to $\boldsymbol{w}''$ and w_0''; since $\boldsymbol{w}'' = \frac{\boldsymbol{w}'}{||\boldsymbol{w}'||_2}$, rearranging for r gives:

$$||\boldsymbol{w}''||_2 = \left\| \frac{\boldsymbol{w}'}{||\boldsymbol{w}'||_2 r} \right\| = \frac{1}{r} \cdot \left\| \frac{\boldsymbol{w}'}{||\boldsymbol{w}'||_2} \right\| = \frac{1}{r}.$$

- by substituting this result into (10), we obtain:

$$\max_{\boldsymbol{w}'', w_0''} \frac{1}{||\boldsymbol{w}''||_2^2}$$
$$\text{subject to } y^{(i)} \left(\boldsymbol{w}''^{\top} \boldsymbol{x}^{(i)} + w_0'' \right) \geq 1, \ 1 \leq i \leq n.$$

- the final step is to observe that maximizing $\frac{1}{||\boldsymbol{w}''||_2^2}$ yields the same solution as minimizing $\frac{1}{2}||\boldsymbol{w}''||_2^2$, which means that the two objectives are equivalent

2.2 Linear SVM classification

- SVMs are sensitive to the feature scales, as we can see in Figure 8: in the left plot, the vertical scale is much larger than the horizontal scale, so the widest possible margin is close to horizontal
- after feature scaling, the decision boundary in the right plot looks much better

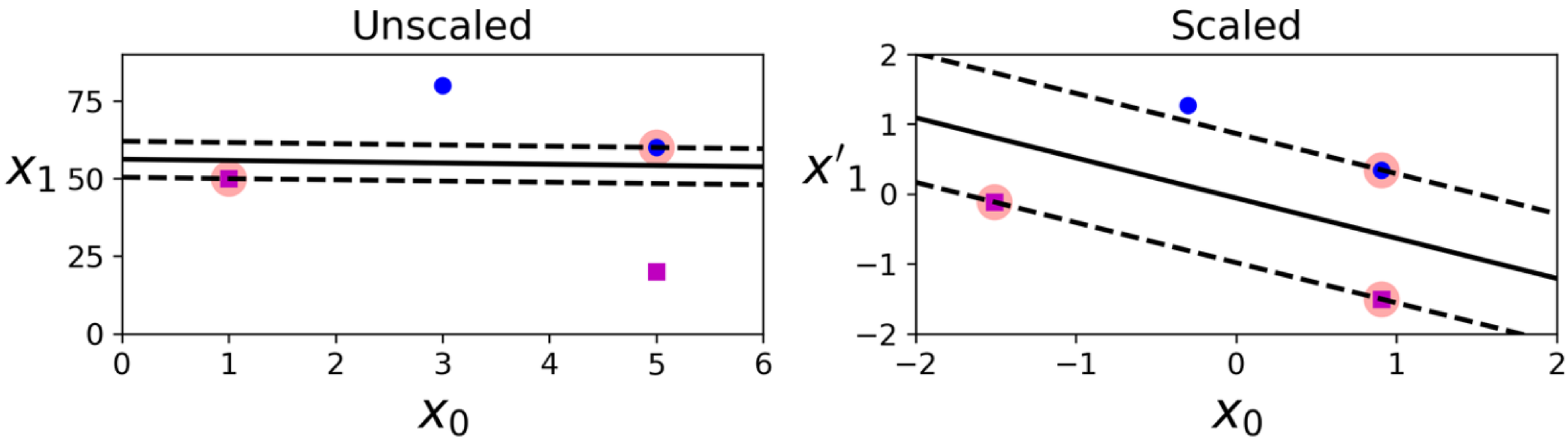


Figure 8: Sensitivity to feature scales.

2.2 Linear SVM classification

- the maximum margin separator is a very natural way to perform classification, if a separating hyperplane *exists*
- however, in many cases, *no separating hyperplane exists*, and so there is no maximum margin separator
- in this case, the optimization problem (5) has no solution with $r > 0$
- an example is shown in Figure 9
- in this case, we cannot *exactly* separate the two classes
- however, as we will see next, we can extend the concept of a separating hyperplane in order to develop a hyperplane that *almost* separates the classes, using a so-called *soft margin*

2.2 Linear SVM classification

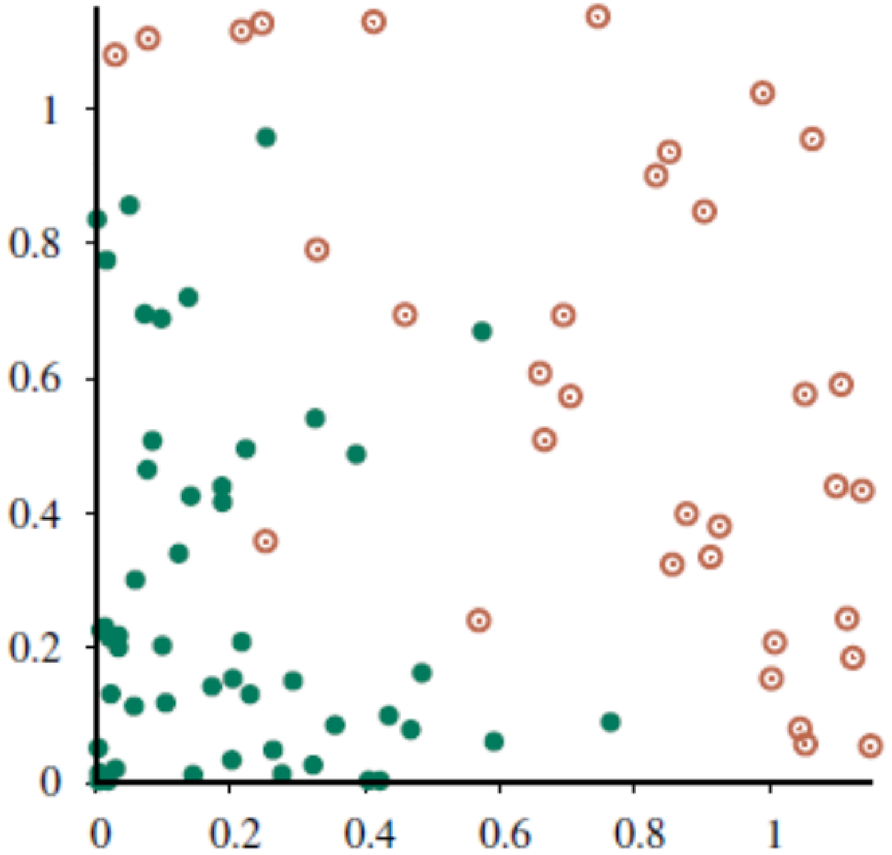


Figure 9: Two classes of points (orange open and green filled circles) that are not separable by a hyperplane.

2.2 Linear SVM classification

- in Figure 9, we see that examples that belong to two classes are not necessarily separable by a hyperplane
- in fact, even if a separating hyperplane does exist, then there are instances in which a classifier based on a separating hyperplane might not be desirable
- a classifier based on a separating hyperplane will necessarily perfectly classify all of the training examples; this can lead to sensitivity to individual examples
- the addition of a single example can lead to a dramatic change in the maximum margin hyperplane
- the resulting maximum margin hyperplane is not satisfactory in this case, because it can have only a tiny margin

2.2 Linear SVM classification

- this is problematic because the distance of an example from the hyperplane can be seen as a measure of our confidence that the example was correctly classified
- moreover, the fact that the maximum margin hyperplane is extremely sensitive to a change in a single example suggests that it may have *overfit* the training data
- in this case, we might be willing to consider a classifier based on a hyperplane that does *not* perfectly separate the two classes, in the interest of:
  - greater robustness to individual examples
  - better classification of *most* of the training examples

2.2 Linear SVM classification

- that is, it could be worthwhile to misclassify a few training examples in order to do a better job in classifying the remaining examples; the *soft margin SVM* does exactly this
- rather than seeking the largest possible margin so that every example is not only on the correct side of the hyperplane but also on the correct side of the margin, we instead allow some examples to be on the incorrect side of the margin, or even the incorrect side of the hyperplane
- in fact, when there is no separating hyperplane, such a situation is inevitable
- the margin is *soft* because it can be *violated* by some of the training examples
- examples on the wrong side of the hyperplane correspond to training examples that are misclassified by the soft margin SVM

2.2 Linear SVM classification

- the soft margin SVM classifies a test example depending on which side of the hyperplane it lies
- the hyperplane is chosen to correctly separate most of the training examples into the two classes, but may misclassify a few examples
- it is the solution to the optimization problem:

$$\begin{aligned} &\max_{\boldsymbol{w}, w_0, \xi^{(1)}, \xi^{(2)}, \ldots, \xi^{(n)}, r} \quad r \\ &\text{subject to } y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq r(1 - \xi^{(i)}),\ \xi^{(i)} \geq 0,\ 1 \leq i \leq n, \\ &\sum_{i=1}^{n} \xi^{(i)} \leq \lambda,\ ||\boldsymbol{w}||_2 = 1,\ r > 0, \end{aligned} \tag{11}$$

  where λ is a nonnegative tuning parameter
- as in (5), r is the width of the margin, so we seek to make this quantity as large as possible
- in (11), $\xi^{(1)}, \xi^{(2)}, \ldots, \xi^{(n)}$ are *slack variables* that allow individual examples to be on the wrong side of the margin or of the hyperplane

2.2 Linear SVM classification

- first of all, the slack variable $\xi^{(i)}$ tells us where the *i*th example is located, relative to the hyperplane and relative to the margin
- if $\xi^{(i)} = 0$, then the *i*th example is on the correct side of the margin
- if $\xi^{(i)} > 0$, then the *i*th example is on the wrong side of the margin, and we say that the *i*th example has *violated* the margin
- if $\xi^{(i)} > 1$, then it is on the wrong side of the hyperplane

2.2 Linear SVM classification

- we now consider the role of the tuning parameter λ; in (11), λ bounds the sum of the $\xi^{(i)}$, and so it determines the number and severity of the violations to the margin (and to the hyperplane) that we will tolerate
- we can think of λ as a *budget* for the amount that the margin can be violated by the n examples
- if $\lambda = 0$, then there is no budget for violations to the margin, and it must be the case that $\xi^{(1)} = \xi^{(2)} = \cdots = \xi^{(n)} = 0$, in which case (11) simply amounts to the maximum margin hyperplane optimization problem (5)
- of course, a maximum margin hyperplane exists only if the two classes are separable
- for $\lambda > 0$, no more than λ examples can be on the wrong side of the hyperplane, because if an example is on the wrong side of the hyperplane, then $\xi^{(i)} > 1$, and (11) requires that $\sum_{i=1}^{n} \xi^{(i)} \leq \lambda$

2.2 Linear SVM classification

- as the budget λ increases, we become more tolerant of violations to the margin, and so the margin will widen
- conversely, as λ decreases, we become less tolerant of violations to the margin and so the margin narrows
- λ controls the bias-variance trade-off of the statistical learning technique
- when λ is small, we seek narrow margins that are rarely violated; this amounts to a classifier that is highly fit to the data, which may have low bias but high variance
- on the other hand, when λ is larger, the margin is wider and we allow more violations to it; this amounts to fitting the data less hard and obtaining a classifier that is potentially more biased, but may have lower variance

2.2 Linear SVM classification

- as for hard margin SVM, only the support vectors affect the classifier, which is in line with our previous assertion that λ controls the bias-variance trade-off of the soft margin SVM
- when the tuning parameter λ is large, then the margin is wide, many examples violate the margin, and so there are many support vectors
- in this case, many examples are involved in determining the hyperplane
- in contrast, if λ is small, then there will be fewer support vectors and hence the resulting classifier will have low bias but high variance
- the fact that the soft margin SVM's decision rule is based only on a potentially small subset of the training examples (the support vectors) means that it is quite robust to the behavior of examples that are far away from the hyperplane

2.2 Linear SVM classification

- corresponding to the optimization formulation (8), the soft margin SVM optimization can be formulated as:

$$\min_{\boldsymbol{w}, w_0, \xi^{(1)}, \xi^{(2)}, \ldots, \xi^{(n)}} \frac{1}{2}||\boldsymbol{w}||_2^2 + C\sum_{i=1}^{n}\xi^{(i)} \tag{12}$$
$$\text{subject to } y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq 1 - \xi^{(i)},\ \xi^{(i)} \geq 0,\ 1 \leq i \leq n.$$

- here, C has the same role as $1/\lambda$ in the previous formulation: a small value of C allows for more margin violations, whereas a large value of C yields a more restrictive classifier
- if $C = \infty$, we recover the hard margin classifier
- the hard margin and soft margin problems are both convex quadratic optimization problems with linear constraints; such problems are known as *quadratic programming (QP)* problems
- many off-the-shelf solvers are available to solve QP problems by using a variety of techniques that are outside the scope of this course

2.2 Linear SVM classification

- we can use gradient descent to minimize a loss function corresponding to the SVM
- but what is the *loss function* corresponding to the SVM?
- consider the error between the output of a predictor $\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0$ and the label $y^{(i)}$
- the loss describes the error that is made on the training data
- an equivalent way to derive (12) is to use the *hinge loss*:

$$\ell(t) = \max\{0, 1 - t\} \text{ where } t = y(\boldsymbol{w}^\top \boldsymbol{x} + w_0). \tag{13}$$

2.2 Linear SVM classification

- if $\boldsymbol{w}^\top \boldsymbol{x} + w_0$ is on the correct side (based on the corresponding label y) of the hyperplane, and further than distance 1, this means that $t > 1$, and the hinge loss returns a value of zero
- if $\boldsymbol{w}^\top \boldsymbol{x} + w_0$ is on the correct side but too close to the hyperplane ($0 < t < 1$), the example $\boldsymbol{x}$ is within the margin, and the hinge loss returns a positive value
- when the example is on the wrong side of the hyperplane ($t < 0$), the hinge loss returns an even larger value, which increases linearly
- in other words, we pay a penalty once we are closer than the margin to the hyperplane, even if the prediction is correct, and the penalty increases linearly

2.2 Linear SVM classification

- an alternative way to express the hinge loss is by considering it as two linear pieces:

$$\ell(t) = \begin{cases} 0, & \text{if } t \geq 1 \\ 1 - t, & \text{if } t < 1 \end{cases}. \tag{14}$$

- the loss corresponding to the hard margin SVM (8) is defined as:

$$\ell(t) = \begin{cases} 0, & \text{if } t \geq 1 \\ \infty, & \text{if } t < 1 \end{cases}.$$

- this loss can be interpreted as never allowing any examples inside the margin

2.2 Linear SVM classification

- for a given training set $\boldsymbol{x}^{(i)}, y^{(i)}$, $1 \leq i \leq n$, we seek to minimize the total loss, while regularizing the objective with ℓ_2-regularization
- using the hinge loss (13) gives us the unconstrained optimization problem:

$$\min_{\boldsymbol{w}, w_0} \frac{1}{2}||\boldsymbol{w}||_2^2 + C\sum_{i=1}^{n} \max\{0, 1 - y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0)\}. \tag{15}$$

- the first term in (15) is called the *regularization term* or the *regularizer*, and the second term is called the *loss term* or the *error term*
- recall that the term $\frac{1}{2}||\boldsymbol{w}||_2^2$ arises directly from the margin; in other words, margin maximization can be interpreted as *regularization*

2.2 Linear SVM classification

- in principle, the unconstrained optimization problem in (15) can be directly solved with subgradient descent methods, because the hinge loss is not derivable at $t = 1$, which means we can use any value between -1 and 0 as the derivative
- to see that (15) and (12) are equivalent, observe that the hinge loss (13) essentially consists of two linear parts, as expressed in (14)
- consider the hinge loss for a single example (13)
- we can equivalently replace minimization of the hinge loss over t with a minimization of a slack variable ξ with two constraints; in equation form:

$$\min_t \max\{0, 1 - t\}$$

is equivalent to

$$\min_{\xi,t} \xi$$

$$\text{subject to } \xi \geq 0,\ \xi \geq 1 - t.$$

- by substituting this expression into (15) and rearranging one of the constraints, we obtain exactly the soft margin SVM (12)

2.2 Linear SVM classification

- the description of the SVM given up until this point, in terms of the variables $\boldsymbol{w}$ and w_0, is known as the *primal SVM*
- recall that we consider inputs $\boldsymbol{x} \in \mathbb{R}^d$ with d features; since $\boldsymbol{w}$ is of the same dimension as $\boldsymbol{x}$, this means that the number of parameters (the dimension of $\boldsymbol{w}$) of the optimization problem grows linearly with the number of features
- in the following, we consider an equivalent optimization problem (the so-called *dual view*), which is independent of the number of features
- instead, the number of parameters increases with the number of examples in the training set
- this is useful for problems where we have more features than the number of examples in the training dataset

2.2 Linear SVM classification

- to understand *duality*, we first need to understand the *Lagrange multipliers* method
- the general idea is to transform a constrained optimization objective into an unconstrained one, by moving the constraints into the objective function
- using the Lagrange multipliers method, we start by defining a new function called the *Lagrangian* (or *Lagrange function*)
- each constraint is subtracted from the original objective, multiplied by a new variable called a *Lagrange multiplier*
- Lagrange showed that a solution to the constrained optimization problem is a *stationary point* of the Lagrangian (a stationary point is a point where all partial derivatives are equal to zero)

2.2 Linear SVM classification

- in other words, we can compute the partial derivatives of the Lagrangian; we can find the points where these derivatives are all equal to zero; and the solutions to the constrained optimization problem (if they exist) must be among these stationary points
- however, this method applies only to equality constraints
- fortunately, under some regularity conditions (which are respected by the SVM objectives), this method can be generalized to inequality constraints as well
- for the *generalized Lagrangian* the new variables are called *Karush–Kuhn–Tucker multipliers*, and they must be greater or equal to zero

2.2 Linear SVM classification

- recall the primal soft margin SVM (12)
- we call the variables $\boldsymbol{w}$, w_0, and $\xi^{(i)}$, $1 \leq i \leq n$, corresponding to the primal SVM the *primal variables*
- we use $\alpha^{(i)}$ as the Karush–Kuhn–Tucker multiplier corresponding to the constraint $y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq 1 - \xi^{(i)}$ that the examples are classified correctly and $\gamma^{(i)} \geq 0$ as the Karush–Kuhn–Tucker multiplier corresponding to the non-negativity constraint of the slack variable, i.e., $\xi^{(i)} \geq 0$, $1 \leq i \leq n$
- the generalized Lagrangian is then given by:

$$\begin{aligned} \mathfrak{L}(\boldsymbol{w}, w_0, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\gamma}) &= \frac{1}{2}||\boldsymbol{w}||_2^2 + C\sum_{i=1}^{n}\xi^{(i)} \\ &\quad - \sum_{i=1}^{n}\alpha^{(i)}[y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) - 1 + \xi^{(i)}] - \sum_{i=1}^{n}\gamma^{(i)}\xi^{(i)}. \end{aligned} \tag{16}$$

2.2 Linear SVM classification

- by differentiating the generalized Lagrangian (16) with respect to the primal variables $\boldsymbol{w}$, w_0, and $\xi^{(i)}$ respectively, we obtain:

$$\frac{\partial \mathfrak{L}}{\partial \boldsymbol{w}} = \boldsymbol{w}^\top - \sum_{i=1}^{n} \alpha^{(i)} y^{(i)} \boldsymbol{x}^{(i)\top}, \tag{17}$$

$$\frac{\partial \mathfrak{L}}{\partial w_0} = -\sum_{i=1}^{n} \alpha^{(i)} y^{(i)}, \tag{18}$$

$$\frac{\partial \mathfrak{L}}{\partial \xi^{(i)}} = C - \alpha^{(i)} - \gamma^{(i)}. \tag{19}$$

- we now find the maximum of the generalized Lagrangian by setting each of these partial derivatives to zero

2.2 Linear SVM classification

- by setting (17) to zero, we find:

$$\boldsymbol{w} = \sum_{i=1}^{n} \alpha^{(i)} y^{(i)} \boldsymbol{x}^{(i)}. \tag{20}$$

- equation (20) states that the optimal weight vector in the primal is a linear combination of the examples $\boldsymbol{x}^{(i)}$
- recall from linear algebra that this means that the solution of the optimization problem lies in the *span* of training data
- additionally, the constraint obtained by setting (18) to zero implies that the optimal weight vector is an affine combination of the examples

2.2 Linear SVM classification

- by substituting the expression for $\boldsymbol{w}$ into the generalized Lagrangian (16), we obtain the dual:

$$\begin{aligned}\mathfrak{D}(\boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\gamma}) \quad = \quad & \frac{1}{2}\sum_{i=1}^{n}\sum_{i'=1}^{n} y^{(i)} y^{(i')} \alpha^{(i)} \alpha^{(i')} \boldsymbol{x}^{(i)\top} \boldsymbol{x}^{(i')} - \sum_{i=1}^{n} \alpha^{(i)} y^{(i)} \left(\sum_{i'=1}^{n} \alpha^{(i')} y^{(i')} \boldsymbol{x}^{(i')} \right)^{\top} \boldsymbol{x}^{(i)} \\ & + C \sum_{i=1}^{n} \xi^{(i)} - w_0 \sum_{i=1}^{n} \alpha^{(i)} y^{(i)} + \sum_{i=1}^{n} \alpha^{(i)} - \sum_{i=1}^{n} \alpha^{(i)} \xi^{(i)} - \sum_{i=1}^{n} \gamma^{(i)} \xi^{(i)}. \qquad (21)\end{aligned}$$

- note that there are no longer any terms involving the primal variable $\boldsymbol{w}$
- by setting (18) to zero, we obtain $\sum_{i=1}^{n} \alpha^{(i)} y^{(i)} = 0$; therefore, the term involving w_0 also vanishes
- the first two terms in (21) are over the same objects; these terms can be simplified, and we obtain the generalized Lagrangian:

$$\mathfrak{D}(\boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\gamma}) = -\frac{1}{2}\sum_{i=1}^{n}\sum_{i'=1}^{n} y^{(i)} y^{(i')} \alpha^{(i)} \alpha^{(i')} \boldsymbol{x}^{(i)\top} \boldsymbol{x}^{(i')} + \sum_{i=1}^{n} \alpha^{(i)} + \sum_{i=1}^{n} (C - \alpha^{(i)} - \gamma^{(i)}) \xi^{(i)}. \qquad (22)$$

2.2 Linear SVM classification

- the last term in this equation is a collection of all terms that contain slack variables $\xi^{(i)}$
- by setting (19) to zero, we see that the last term in (22) is also zero
- furthermore, by using the same equation and recalling that the Karush–Kuhn–Tucker multipliers $\gamma^{(i)}$ are non-negative, we conclude that $\alpha^{(i)} \leq C$
- we now obtain the dual optimization problem of the SVM, which is expressed exclusively in terms of the Karush–Kuhn–Tucker multipliers $\alpha^{(i)}$
- in the Lagrangian duality setting, we have to maximize the dual problem
- this is equivalent to minimizing the negative dual problem, such that we end up with the *dual SVM*:

$$\begin{aligned} \min_{\alpha^{(1)},\alpha^{(2)},\ldots,\alpha^{(n)}} \; & \frac{1}{2}\sum_{i=1}^{n}\sum_{i'=1}^{n} y^{(i)} y^{(i')} \alpha^{(i)} \alpha^{(i')} \boldsymbol{x}^{(i)\top} \boldsymbol{x}^{(i')} - \sum_{i=1}^{n} \alpha^{(i)} \\ \text{subject to } & \sum_{i=1}^{n} \alpha^{(i)} y^{(i)} = 0 \\ & 0 \leq \alpha^{(i)} \leq C, \; 1 \leq i \leq n. \end{aligned} \tag{23}$$

2.2 Linear SVM classification

- once we obtain the dual parameters $\alpha^{(i)}$, we can recover the primal parameters $\boldsymbol{w}$ by using (20)
- let us call the optimal primal parameter $\boldsymbol{w}^*$; however, there remains the question on how to obtain the parameter w_0^*
- consider an example $\boldsymbol{x}^{(i)}$ that lies exactly on the margin's boundary, i.e., $\boldsymbol{w}^{*\top}\boldsymbol{x}^{(i)} + w_0 = y^{(i)}$
- recall that $y^{(i)}$ is either $+1$ or -1; therefore, the only unknown is w_0^*, which can be computed by:

$$w_0^* = y^{(i)} - \boldsymbol{w}^{*\top}\boldsymbol{x}^{(i)}.$$

2.2 Linear SVM classification

- however, it is often preferred to compute the average over all support vectors to get a more stable and precise value, as in:

$$w_0^* = \frac{1}{n_s} \sum_{\substack{i=1 \\ \boldsymbol{w}^{*\top}\boldsymbol{x}^{(i)}+w_0=y^{(i)}}}^{n} y^{(i)} - \boldsymbol{w}^{*\top}\boldsymbol{x}^{(i)},$$

  where n_s is the number of support vectors
- in conclusion, the primal parameters can be computed as:

$$\begin{aligned} \boldsymbol{w}^* &= \sum_{i=1}^{n} \alpha^{(i)} y^{(i)} \boldsymbol{x}^{(i)} \\ w_0^* &= \frac{1}{n_s} \sum_{\substack{i=1 \\ \boldsymbol{w}^{*\top}\boldsymbol{x}^{(i)}+w_0=y^{(i)}}}^{n} y^{(i)} - \boldsymbol{w}^{*\top}\boldsymbol{x}^{(i)}. \end{aligned} \tag{24}$$

2.3 Nonlinear SVM classification

- although linear SVM classifiers are efficient and work surprisingly well in many cases, many datasets are not even close to being linearly separable
- one approach to handling nonlinear datasets is to add more features, such as polynomial features; in some cases this can result in a linearly separable dataset

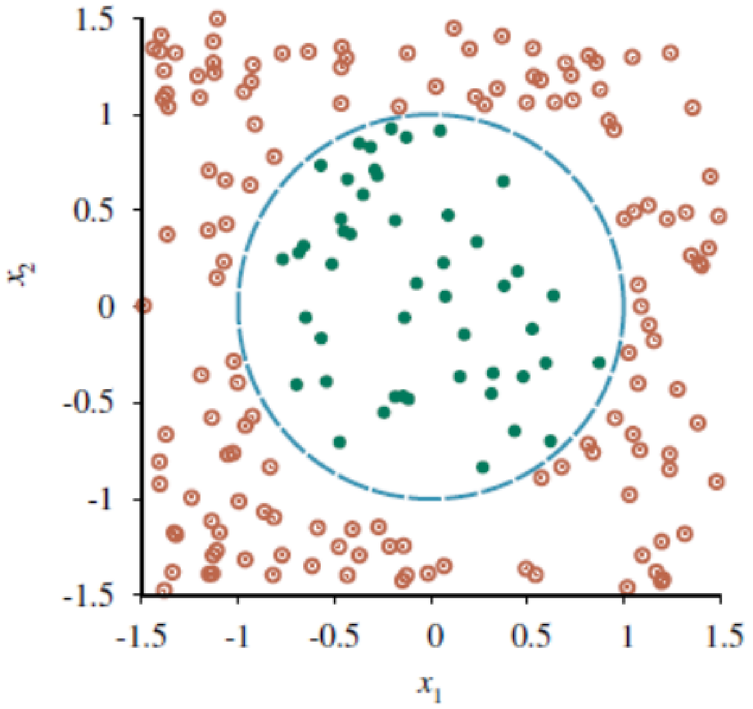


Figure 10: A two-dimensional training set with positive examples as green filled circles and negative examples as orange open circles. The true decision boundary, $x_1^2 + x_2^2 \leq 1$, is also shown.

2.3 Nonlinear SVM classification

- Figure 10 shows an input space defined by attributes $\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, with positive examples ($y = +1$) inside a circular region and negative examples ($y = -1$) outside
- clearly, there is no linear separator for this problem
- now, suppose we re-express the input data – that is, we map each input vector $\boldsymbol{x}$ to a new vector of feature values, $\phi(\boldsymbol{x})$
- in particular, let us use the three features:

$$\phi(\boldsymbol{x}) = \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}x_1 x_2 \end{bmatrix}. \tag{25}$$

2.3 Nonlinear SVM classification

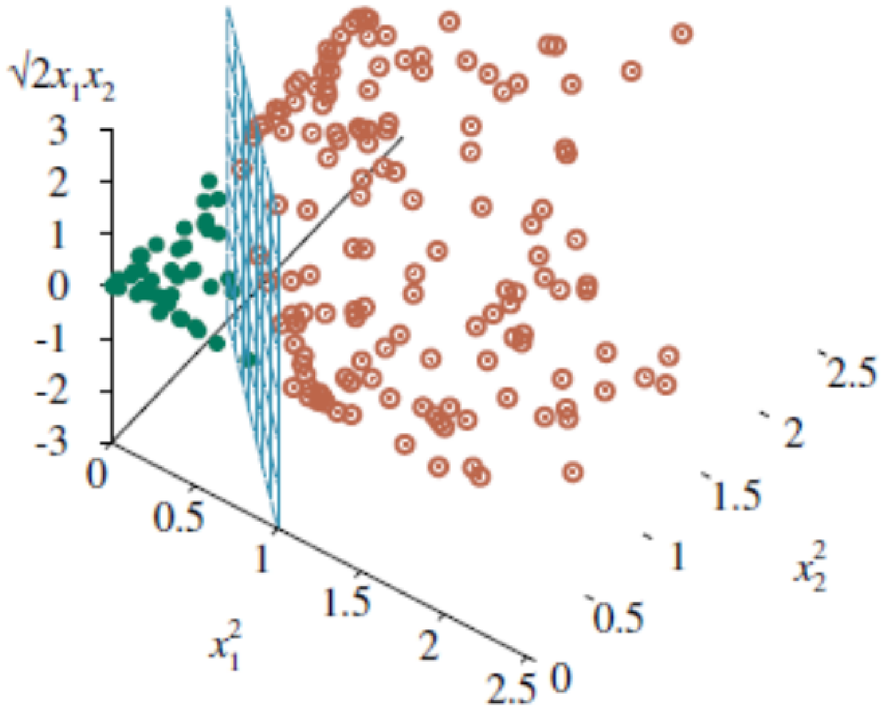


Figure 11: The same data after mapping into a three-dimensional input space $\begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}x_1x_2 \end{bmatrix}$. The circular decision boundary in Figure 10 becomes a linear decision boundary in three dimensions.

2.3 Nonlinear SVM classification

- we will see shortly where these came from, but for now, just look at what happens
- Figure 11 shows the data in the new, three-dimensional space defined by the three features; the data are *linearly separable* in this space!
- this phenomenon is actually fairly general: if data are mapped into a space of sufficiently high dimension, then they will almost always be linearly separable – if we look at a set of points from enough directions, we'll find a way to make them line up
- in general (with some special cases excepted) if we have n data points, then they will always be separable in spaces of $n - 1$ dimensions or more

2.3 Nonlinear SVM classification

- adding polynomial features is simple to implement and can work great with all sorts of machine learning algorithms (not just SVMs)
- that said, at a low polynomial degree, this method cannot deal with very complex datasets, and with a high polynomial degree it creates a huge number of features, making the model too slow
- fortunately, when using SVMs, we can apply an almost miraculous mathematical technique called the *kernel trick*
- the kernel trick makes it possible to get the same result as if we had added many polynomial features, even with very high-degree polynomials, without actually having to add them
- so there is no combinatorial explosion of the number of features because we don't actually add any features

2.3 Nonlinear SVM classification

- now, let's look at what happens to a couple of 2D vectors, $\boldsymbol{a}$ and $\boldsymbol{b}$, if we apply the second-degree polynomial mapping $\phi(\boldsymbol{x})$, and then compute the dot product of the transformed vectors:

$$\begin{aligned}
\phi(\boldsymbol{a})^\top \phi(\boldsymbol{b}) &= \begin{bmatrix} a_1^2 \\ a_2^2 \\ \sqrt{2}a_1 a_2 \end{bmatrix}^\top \begin{bmatrix} b_1^2 \\ b_2^2 \\ \sqrt{2}b_1 b_2 \end{bmatrix} \\
&= a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2 \\
&= (a_1 b_1 + a_2 b_2)^2 \\
&= \left(\begin{bmatrix} a_1 \\ a_2 \end{bmatrix}^\top \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} \right)^2 \\
&= (\boldsymbol{a}^\top \boldsymbol{b})^2.
\end{aligned}$$

- we observe that the dot product of the transformed vectors is equal to the square of the dot product of the original vectors: $\phi(\boldsymbol{a})^\top \phi(\boldsymbol{b}) = (\boldsymbol{a}^\top \boldsymbol{b})^2$

2.3 Nonlinear SVM classification

- here is the key insight: if we apply the transformation ϕ to all training instances, then the dual problem in (23) will contain the dot product $\phi(\boldsymbol{x}^{(i)})^\top \phi(\boldsymbol{x}^{(i')})$
- but if ϕ is the second-degree polynomial transformation defined in (25), then we can replace this dot product of transformed vectors simply by $\left(\boldsymbol{x}^{(i)\top} \boldsymbol{x}^{(i')}\right)^2$
- so, we don't need to transform the training instances at all; just replace the dot product by its square in (23)
- the result will be strictly the same as if we had gone through the trouble of transforming the training set then fitting a linear SVM algorithm, but this trick makes the whole process much more computationally efficient

2.3 Nonlinear SVM classification

- the function $K(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{a}^\top \boldsymbol{b})^2$ is a second-degree polynomial kernel
- in machine learning, a *kernel* is a function capable of computing the dot product $\phi(\boldsymbol{a})^\top \phi(\boldsymbol{b})$, based only on the original vectors $\boldsymbol{a}$ and $\boldsymbol{b}$, without having to compute (or even to know about) the transformation ϕ
- it represents a *generalization* of the *dot product*
- we list some of the most commonly used kernels:

$$\begin{aligned}
\text{Linear:} \quad & K(\boldsymbol{a}, \boldsymbol{b}) = \boldsymbol{a}^\top \boldsymbol{b} \\
\text{Polynomial:} \quad & K(\boldsymbol{a}, \boldsymbol{b}) = (\gamma \boldsymbol{a}^\top \boldsymbol{b} + \beta)^p \\
\text{Gaussian RBF:} \quad & K(\boldsymbol{a}, \boldsymbol{b}) = \exp\left(-\gamma ||\boldsymbol{a} - \boldsymbol{b}||_2^2\right) \\
\text{Sigmoid:} \quad & K(\boldsymbol{a}, \boldsymbol{b}) = \tanh(\gamma \boldsymbol{a}^\top \boldsymbol{b} + \beta).
\end{aligned}$$

2.3 Nonlinear SVM classification

- in the Gaussian RBF kernel, γ is a positive constant
- the Gaussian RBF works as follows: if a given test example $\boldsymbol{x}^{(t)}$ is far from a training example $\boldsymbol{x}^{(i)}$ in terms of Euclidean distance, then $||\boldsymbol{x}^{(t)} - \boldsymbol{x}^{(i)}||_2^2$ will be large, and so $K(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(i)}) = \exp\left(-\gamma||\boldsymbol{x}^{(t)} - \boldsymbol{x}^{(i)}||_2^2\right)$ will be tiny
- this means that $\boldsymbol{x}^{(i)}$ will play virtually no role in $\boldsymbol{w}^{*\top}\phi(\boldsymbol{x}^{(t)}) + w_0^*$
- recall that the predicted class label for the test example $\boldsymbol{x}^{(t)}$ is based on the sign of $\boldsymbol{w}^{*\top}\phi(\boldsymbol{x}^{(t)}) + w_0^*$
- in other words, training examples that are far from $\boldsymbol{x}^{(t)}$ will play essentially no role in the predicted class label for $\boldsymbol{x}^{(t)}$
- this means that the radial kernel has very *local* behavior, in the sense that only nearby training examples have an effect on the class label of a test example

2.3 Nonlinear SVM classification

- according to *Mercer's theorem*, if a function $K(\boldsymbol{a}, \boldsymbol{b})$ respects a few mathematical conditions called *Mercer's conditions* (e.g., K must be continuous and symmetric in its arguments so that $K(\boldsymbol{a}, \boldsymbol{b}) = K(\boldsymbol{b}, \boldsymbol{a})$, etc.), then there exists a function ϕ that maps $\boldsymbol{a}$ and $\boldsymbol{b}$ into another space (possibly with much higher dimensions) such that $K(\boldsymbol{a}, \boldsymbol{b}) = \phi(\boldsymbol{a})^\top \phi(\boldsymbol{b})$
- we can use K as a kernel because we know ϕ exists, even if we don't know what ϕ is
- in the case of the Gaussian RBF kernel, it can be shown that ϕ maps each training instance to an infinite-dimensional space, so it's a good thing we don't need to actually perform the mapping!
- note that some frequently used kernels (such as the sigmoid kernel) don't respect all of Mercer's conditions, yet they generally work well in practice

2.3 Nonlinear SVM classification

- equations (24) show how to go from the dual solution to the primal solution for a linear SVM classifier; but if we apply the kernel trick, we end up with equations that include $\phi(\boldsymbol{x}^{(i)})$
- in fact, $\boldsymbol{w}^*$ must have the same number of dimensions as $\phi(\boldsymbol{x}^{(i)})$, which may be huge or even infinite, so we can't compute it
- in order to make predictions without knowing $\boldsymbol{w}^*$, we can plug the formula for $\boldsymbol{w}^*$ from equations (24) into the decision function for a new instance $\boldsymbol{x}^{(t)}$, and we get an equation with only dot products between input vectors
- this makes it possible to use the kernel trick:

$$\begin{aligned}
\boldsymbol{w}^{*\top}\phi(\boldsymbol{x}^{(t)}) + w_0^* &= \left(\sum_{i=1}^{n} \alpha^{(i)} y^{(i)} \phi(\boldsymbol{x}^{(i)})\right)^{\top} \phi(\boldsymbol{x}^{(t)}) + w_0^* \\
&= \sum_{i=1}^{n} \alpha^{(i)} y^{(i)} \phi(\boldsymbol{x}^{(i)})^{\top} \phi(\boldsymbol{x}^{(t)}) + w_0^* \\
&= \sum_{\substack{i=1 \\ \boldsymbol{w}^{*\top}\phi(\boldsymbol{x}^{(i)})+w_0=y^{(i)}}}^{n} \alpha^{(i)} y^{(i)} K(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(t)}) + w_0^*. \qquad (26)
\end{aligned}$$

2.3 Nonlinear SVM classification

- note that since $\boldsymbol{w}^{*\top}\phi(\boldsymbol{x}^{(i)}) + w_0 = y^{(i)}$ only for support vectors, making predictions involves computing the dot product of the new input vector $\boldsymbol{x}^{(t)}$ with only the support vectors, not all the training instances
- of course, we need to use the same trick to compute the bias term:

$$
\begin{aligned}
w_0^* &= \frac{1}{n_s} \sum_{\substack{i=1 \\ \boldsymbol{w}^{*\top}\phi(\boldsymbol{x}^{(i)})+w_0=y^{(i)}}}^{n} y^{(i)} - \boldsymbol{w}^{*\top}\phi(\boldsymbol{x}^{(i)}) \\
&= \frac{1}{n_s} \sum_{\substack{i=1 \\ \boldsymbol{w}^{*\top}\phi(\boldsymbol{x}^{(i)})+w_0=y^{(i)}}}^{n} y^{(i)} - \left(\sum_{i'=1}^{n} \alpha^{(i')} y^{(i')} \phi(\boldsymbol{x}^{(i')})\right)^{\top} \phi(\boldsymbol{x}^{(i)}) \\
&= \frac{1}{n_s} \sum_{\substack{i=1 \\ \boldsymbol{w}^{*\top}\phi(\boldsymbol{x}^{(i)})+w_0=y^{(i)}}}^{n} y^{(i)} - \sum_{\substack{i'=1 \\ \boldsymbol{w}^{*\top}\phi(\boldsymbol{x}^{(i')})+w_0=y^{(i')}}}^{n} \alpha^{(i')} y^{(i')} K(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(i')}).
\end{aligned}
$$

2.3 Nonlinear SVM classification

- so far, our discussion has been limited to the case of binary classification: that is, classification in the two-class setting
- how can we extend SVMs to the more general case, where we have an arbitrary number of classes?
- it turns out that the concept of *separating hyperplanes* upon which SVMs are based does not lend itself naturally to more than two classes
- though a number of proposals for extending SVMs to the q-class case have been made, the two most popular are the *one-versus-one* and *one-versus-all* approaches

2.3 Nonlinear SVM classification

- suppose that we would like to perform classification using SVMs, and there are $q > 2$ classes
- a *one-versus-one* or *all-pairs* approach constructs $C_q^2 = \frac{q(q-1)}{2}$ SVMs, each of which compares a pair of classes
- for example, one such SVM might compare the kth class, coded as $+1$, to the k'th class, coded as -1
- we classify a test example using each of the $C_q^2 = \frac{q(q-1)}{2}$ classifiers, and we calculate the total number of times that the test example is assigned to each of the q classes
- the final classification is performed by assigning the test example to the class to which it was most frequently assigned in these $C_q^2 = \frac{q(q-1)}{2}$ pairwise classifications

2.3 Nonlinear SVM classification

- the *one-versus-all* approach is an alternative procedure for applying SVMs in the case of $q > 2$ classes
- we fit q SVMs, each time comparing one of the q classes to the remaining $q - 1$ classes
- let $\boldsymbol{w}_k$ denote the parameters that result from fitting an SVM comparing the kth class (coded as $+1$) to the others (coded as -1)
- let $\boldsymbol{x}^{(t)}$ denote a test example; we assign the example to the class for which $\boldsymbol{w}_k^\top \boldsymbol{x}^{(t)}$ is largest, as this amounts to a high level of confidence that the test example belongs to the kth class rather than to any of the other classes

2.4 SVM regression

- as mentioned earlier, the SVM algorithm is versatile: not only does it support linear and nonlinear classification, but it also supports linear and nonlinear regression
- to use SVMs for regression instead of classification, the trick is to reverse the objective: instead of trying to fit the largest possible margin between two classes, while limiting margin violations, SVM regression tries to fit as many instances as possible *inside* the margin, while limiting margin violations (i.e., instances *outside* the margin)
- the width of the margin is controlled by a hyperparameter, ϵ

2.4 SVM regression

- Figure 12 shows two linear SVM regression models trained on some random linear data, one with a large margin ($\epsilon = 1.5$) and the other with a small margin ($\epsilon = 0.5$)

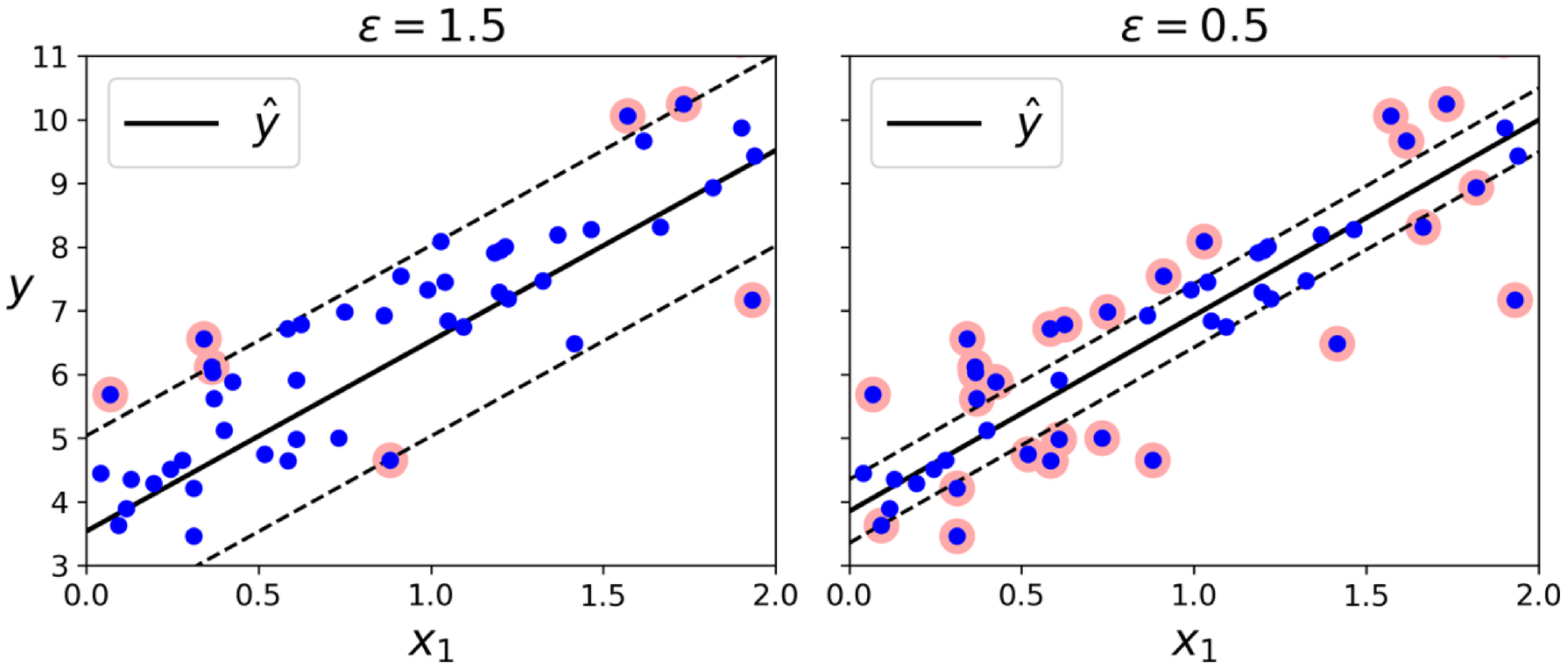


Figure 12: SVM regression.

2.4 SVM regression

- adding more training instances within the margin does not affect the model's predictions; thus, the model is said to be ϵ-*insensitive*

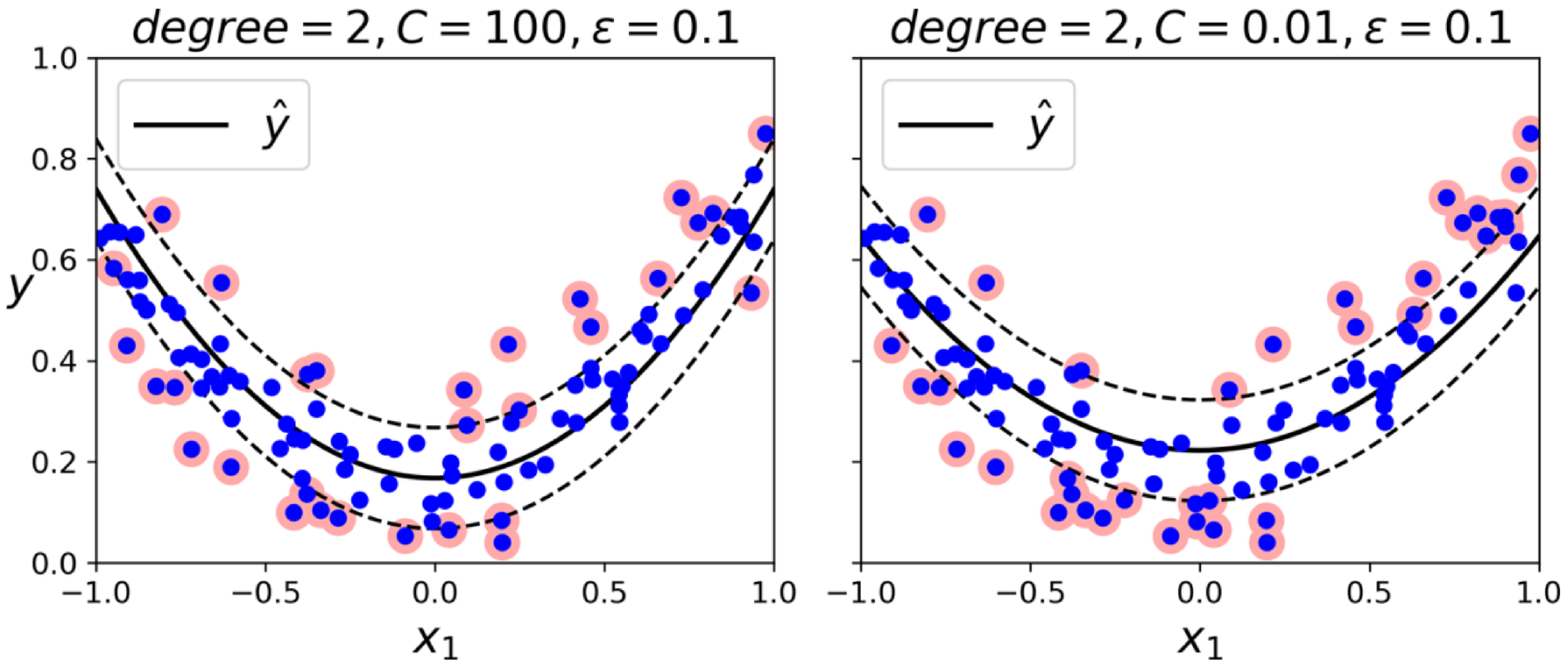


Figure 13: SVM regression using a second-degree polynomial kernel.

2.4 SVM regression

- to tackle nonlinear regression tasks, we can use a kernelized SVM model
- Figure 13 shows SVM regression on a random quadratic training set, using a second-degree polynomial kernel
- there is little regularization in the left plot (i.e., a large C value), and much more regularization in the right plot (i.e., a small C value)

2.4 SVM regression

- in terms of computational complexity, the linear SVM scales almost linearly with the number of training instances and the number of features
- its training time complexity is roughly $\mathcal{O}(nd)$
- applying gradient descent also has a complexity of $\mathcal{O}(nd)$
- the algorithm takes longer if we require very high precision; this is controlled by the tolerance hyperparameter ϵ, and, in most classification tasks, the default tolerance is fine

2.4 SVM regression

- for nonlinear SVM, which supports the kernel trick, the training time complexity is usually between $\mathcal{O}(n^2 d)$ and $\mathcal{O}(n^3 d)$
- unfortunately, this means that it gets very slow when the number of training instances gets large (e.g., hundreds of thousands of instances)
- this algorithm is perfect for complex small- or medium-sized training sets
- it scales well with the number of features, especially with *sparse features* (i.e., when each instance has few nonzero features)
- in this case, the algorithm scales roughly with the average number of nonzero features per instance

2.4 SVM regression

- kernelized SVMs are powerful models and perform well on a variety of datasets
- SVMs allow for complex decision boundaries, even if the data has only a few features
- they work well on low-dimensional and high-dimensional data (i.e., few and many features), but don't scale very well with the number of examples
- running an SVM on data with up to $10,000$ examples might work well, but working with datasets of size $100,000$ or more can become challenging in terms of runtime and memory usage

2.4 SVM regression

- another downside of SVMs is that they require careful preprocessing of the data and tuning of the parameters
- this is why, these days, most people instead use tree-based models such as random forests or gradient boosting (which require little or no preprocessing) in many applications
- furthermore, SVM models are hard to inspect; it can be difficult to understand why a particular prediction was made, and it might be tricky to explain the model to a nonexpert
- still, it might be worth trying SVMs, particularly if all of our features represent measurements in similar units and they are on similar scales

Exercises

Exercise 2

Assume we have a training set for two-class classification in one dimension that contains three sample points: point $x^{(1)} = 3$ with label $y^{(1)} = 1$, point $x^{(2)} = 1$ with label $y^{(2)} = 1$, and point $x^{(3)} = -1$ with label $y^{(3)} = -1$. What are the values of w and w_0 given by a hard-margin SVM?

Solution:

- hard-margin SVM:

$$\max_{\boldsymbol{w}, w_0, r} \; r$$

$$\text{subject to } y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq r, \; 1 \leq i \leq n, \; ||\boldsymbol{w}||_2 = 1, \; r > 0$$

Exercises

- in our case, we have that

$$\begin{aligned} y^{(1)}(wx^{(1)} + w_0) &\geq r \\ y^{(2)}(wx^{(2)} + w_0) &\geq r \\ y^{(3)}(wx^{(3)} + w_0) &\geq r \end{aligned}$$

which is equivalent with

$$\begin{aligned} 1(3w + w_0) &\geq r \\ 1(1w + w_0) &\geq r \\ -1(-1w + w_0) &\geq r \end{aligned}$$

Exercises

which is equivalent with

$$\begin{aligned} 3w + w_0 &\geq r \\ w + w_0 &\geq r \\ w - w_0 &\geq r \end{aligned}$$

- from

$$\begin{aligned} w + w_0 &\geq r \\ w - w_0 &\geq r \end{aligned}$$

we have

$$w \geq r$$

- we want to maximize r, so we take $r = w$

Exercises

- now,

$$\begin{aligned} 3w + w_0 &\geq r \\ w + w_0 &\geq r \\ w - w_0 &\geq r \end{aligned}$$

becomes

$$\begin{aligned} 3w + w_0 &\geq w \\ w + w_0 &\geq w \\ w - w_0 &\geq w \end{aligned}$$

Exercises

which is equivalent with

$$\begin{aligned} 2w &\geq -w_0 \\ w_0 &\geq 0 \\ -w_0 &\geq 0 \end{aligned}$$

from where $w_0 = 0$ and $w \geq 0$

- but $||\boldsymbol{w}||_2 = 1$ so $w = 1$
- thus, $w = 1$ and $w_0 = 0$

Vă mulţumesc!